

group_project1

November 25, 2025

0.1 COMP5712M: Programming for Data Science

0.2 Group project: Data Analysis Project

1 *Data Science for Good: Kiva Crowdfunding Analysis*

- Kaustubh Kaushal, wftn0343@leeds.ac.uk
- Siddhant Ram, fmtm0110@leeds.ac.uk
- Vaishali Nainwal, shtd0023@leeds.ac.uk
- Yash Rajabhau Ingle, rzsg0141@leeds.ac.uk

2 Project Plan

2.1 The Data

1. Source of the Dataset _The data used in this project comes from Kiva, a global microfinance platform that provides crowdfunding loans to low-income individuals. Each row in this dataset represents a real borrower such as a farmer seeking to buy seeds, a mother expanding her shop or a student paying tuition. A cleaned and merged version of Kiva's dataset was taken from Kaggle and further refined. The dataset covers loans posted between 2005 and 2017, capturing over a decade of global microfinance activity across 80+ countries.

Kiva. 2018. *Kiva Crowdfunding Dataset (Data Science for Good)*. [online]. [Accessed 17 November 2025]. Available from: <https://www.kaggle.com/datasets/kiva/data-science-for-good-kiva-crowdfunding>

2. Dataset Structure and Contents *The project uses three interconnected datasets, each representing a different dimension of global microfinance activity. Together, they showcase detailed information about the financial needs of borrowers, the lending response and the purpose of the loan requirement.*

I. Main Loan Data - kiva_loans.csv

This primary dataset contains loan level attributes:

Key components: * **Financial variables:** loan_amount, funded_amount, lender_count, term_in_months * **Borrower and sector attributes:** sector, activity, borrower_genders, country * **Time based variables:** posted_time, funded_time and disbursed_time * **Partner information:** partner_id (linking loan to its field partner)

II. Socioeconomic Context - kiva_mpi_region_locations.csv

This dataset connects the loan records with Multidimensional Poverty Index (MPI) values to reflect on region specific poverty.

Key components: * **Country and region identifiers** to allow mapping with the main dataset.
* **MPI** score (0 - 1) that provides levels of deprivation across health, education and standard of living.

III. Development Theme Mapping - loan_theme_ids.csv

The third dataset maps loan to development themes, such as women's empowerment, education, green energy, post-conflict rehabilitation or rural development.

Key components: * **Loan_id**, connecting directly to the main dataset. * **Theme_id**, indicating development focus.

3. Data Cleaning and Quality *Before analysis, the Kiva dataset required careful cleaning. Several loans had missing or inconsistent timestamps, gender labels and country names that did not match MPI records.*

In order to ensure the reliability of these variables, the cleaning process focused on below aspects:

- Standardizing text values (like loan activities, country names, sectors).
- Removing non-matching nations.
- Median imputation for missing durations and inconsistent date format.

These steps, reshaped the raw data into a high-quality, coherent dataset suitable for robust exploratory analysis and machine-learning modelling.

4. Why This Dataset Is Appropriate *The Kiva microfinance dataset is a powerful fit for this project because it combines: 1. **Loan Behaviour** (funding patterns, lending speed, lender engagement) 2. **Socioeconomic Vulnerability** through MPI 3. **Development Themes** that reflect the purpose and impact of microfinance*

Together, these characteristics allow the project to meet all core coursework requirements:

- Global and regional exploratory analysis
- Socioeconomic and inequality analysis
- Predictive modelling using classification
- Geospatial and time-series analysis of funding behaviour

5. Data Accuracy and Limitations *The Kiva's reporting is generally consistent, mainly its financial fields like loan amount, fund amount and lender participation. The MPI scores provide valuable socioeconomic context which helps us analyse the vulnerability across regions.*

However, limitations exist:

- Some regions lack full MPI coverage
- Borrower attributes (gender, activity descriptions) may be inconsistent
- Several timestamps are missing or misaligned

These limitations do not significantly weaken the dataset but they emphasise the need for careful data cleaning and responsible interpretation.

2.2 Project Aim and Objectives

The aim of this project is to explore how global microfinance patterns work by analysing the Kiva’s crowdfunding loan dataset. Unlike traditional loans, Kiva offers loans to people based on their stories, needs and social impact. This makes the dataset uniquely valuable for understanding why some communities receive faster or more successful financial support than others.

At its core, the project investigates how the borrower’s characteristics (such as gender, sector and loan purpose), regional inequality and poverty shape the outcomes of whether a loan gets fully funded and how quickly it gets approved.

A major part of analysis showcases loan behaviour with the Multidimensional Poverty Index (MPI), allowing us to study what gets funded & why certain regions face deprivation.

The project also builds a predictive model to estimate chances of receiving a fully funded loan and expected funding duration. These models show how a data driven approach can help support microfinance platforms to identify loan risks early, improve transparency and give support to ones in most need.

Overall, this project delivers a complete analytical pipeline—from data cleaning and integration to modelling and interpretation—designed to explain the complex dynamics behind global microlending and the social factors that shape it.

2.2.1 Specific Objective(s)

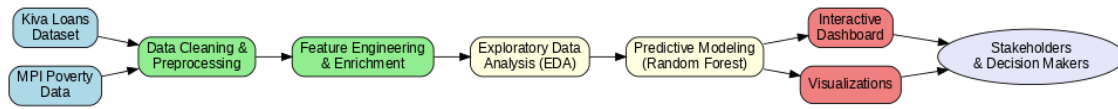
- **Objective 1:** Perform detailed exploratory analysis of global loan patterns, including geographic distribution, sector trends, borrower demographics, and funding behaviour.
- **Objective 2:** Identify and quantify correlations between borrower attributes, loan characteristics, socioeconomic vulnerability (MPI), and funding outcomes.
- **Objective 3:** Build predictive machine learning models to estimate (a) probability of full funding (classification) and (b) expected funding duration (regression).
- **Objective 4:** Develop interactive visualisations and dashboards that support clear, accessible interpretation of global microfinance patterns and model predictions.

2.3 System Design

2.3.1 Architecture

In order to process and analyze the Kiva loans dataset enhanced with Multidimensional Poverty Index (MPI) data, our project uses a modular pipeline architecture. To guarantee a clear, reliable, and perceptive analysis, the data is sequentially processed through distinct stages:

- * **Data Acquisition and Cleaning:** To address missing values, inconsistent entries, and outliers, raw datasets are imported and cleaned.
- * **Feature Engineering and Enrichment:** To contextualize loans, external socioeconomic data (MPI) is combined with existing attributes (such as loan duration and gender bucketing) to create new features.
- * **Exploratory Data Analysis (EDA):** Data patterns and correlations are revealed through statistical summarization and visualization.
- * **Predictive modeling:** Machine learning algorithms, mainly Random Forest classifiers and regressors, forecast loan funding durations and success.
- * **Visualization and Interactive Dashboard:** Stakeholder exploration and decision-making are facilitated by the presentation of results through both static and interactive charts.



2.3.2 Processing Modules and Algorithms

Data Cleaning & Standardization * Purpose: Correct inconsistencies, handle missing values, standardize text fields, and align country names with MPI data. * Techniques: Median imputation for missing durations, regex-based country filtering, categorical consolidation for borrower genders. * Challenges: Handling irregular timestamp formats, missing or invalid entries, and non-matching country-region pairs.

Data Enrichment * Purpose: Combine multiple datasets (kiva_loans.csv, kiva_mpi_region_locations.csv, loan_theme_ids.csv) to create a richer analytical dataset. * Techniques: Merging on loan ID and country, aggregating themes using groupby, mapping MPI regions to loans. * Challenges: Managing many-to-one relationships, handling missing theme information, and preserving categorical types.

Feature Engineering * Purpose: Derive new predictive and analytical features such as `funding_duration_days`, `gender_bucket`, and binary funded labels. * Techniques: Date arithmetic for funding duration, custom function for gender categorization, and one-hot encoding for categorical features. * Challenges: Dealing with zero or negative durations, maintaining category consistency across training and test sets.

Exploratory Data Analysis (EDA) * Purpose: Identify patterns and trends in loan performance, sectoral differences, and geographic distribution. * Techniques: Summary statistics, correlation heatmaps, pivot tables, and interactive visualizations using Plotly (choropleths, treemaps, bar charts). * Challenges: Handling large datasets efficiently, ensuring readability in plots, and enabling comparisons across multiple categorical dimensions.

Predictive Modeling * Purpose: Predict funding success (classification) and funding duration (regression). * Techniques: Random Forest Classifier and Regressor; feature scaling via `RobustScaler`; categorical encoding via `OneHotEncoder`; performance evaluated with ROC-AUC (classification) and RMSE (regression). * Challenges: Avoiding overfitting with high-cardinality categorical variables, combining numeric and categorical features, and maintaining model interpretability.

Interactive Dashboard * Purpose: Provide real-time filtering, visualization, and model prediction exploration for users. * Techniques: IPython widgets for selection, Plotly for dynamic charts, on-the-fly predictions using trained models. * Challenges: Balancing responsiveness with dataset size and integrating predictions seamlessly into the dashboard.

3 Program Code

Brief Explanation of cell 1 This cell imports all the essential libraries which are required to run this project. This includes tools for manipulation (`pandas`, `numpy`), visualization (`matplotlib`, `seaborn`, `plotly`), interactive widgets (`ipywidgets`), model building (`scikit-learn`), and display functions. It also suppresses unnecessary warnings and applies a clean visual theme to ensure consistent environment throughout the notebook.

```
[1]: # Imports & global config
import warnings                                # Handles and suppresses warning messages
    ↳during execution
warnings.filterwarnings('ignore')              #Turns off all unnecessary warning output
    ↳in the notebook
from pathlib import Path
import pandas as pd, numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express as px
import ipywidgets as widgets                  # Creates interactive UI elements
    ↳in Jupyter notebooks
from IPython.display import display, HTML      # Displays HTML, widgets, and rich
    ↳content in Jupyter
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, RobustScaler
from sklearn.ensemble import RandomForestClassifier, RandomForestRegressor
from sklearn.metrics import roc_auc_score, roc_curve
sns.set(style='whitegrid')
```

Brief Explanation of cell 2 This cell shows the file paths and directories which are used throughout the project. It specifies where the raw datasets and cleaned dataset are saved, and sets up an output folder for storing analysis results. Using Path ensures reliable and readable file-path handling across the notebook. The output directory is created if it doesn't already exist, preventing errors during file writing.

```
[2]: # Parameters / paths
DATA = Path(r"C:\Users\orste\OneDrive - University of Leeds\Desktop\Prog for DS\
    ↳Pracs\Group_Project") # Here Path, Simplifies working with file and
    ↳directory paths
RAW_FP = DATA / 'kiva_loans.csv'
MPI_FP = DATA / 'kiva_mpi_region_locations.csv'
CLEAN_FP = DATA / 'kiva_loans_clean_final.csv'
OUT = DATA / 'analysis_outputs'
OUT.mkdir(exist_ok=True)
```

This cell ensures that the output directory exists so that subsequent analyses can save results without errors.

Brief Explanation of cell 3 This cell defines a set of utility functions to clean and standardize the dataset. The first two helpers safely convert selected columns to numeric or datetime formats, coercing invalid values to NaN to prevent errors during analysis. The `gender_bucket_val` function processes the borrower gender field and categorizes each entry into standard groups (female, male, mixed, or unknown) ensuring consistent gender classification for cleaner analysis.

```
[3]: # Utility functions
def safenum(df, cols):
    for c in cols: df[c] = pd.to_numeric(df[c], errors='coerce')

def safedatetime(df, cols):
    for c in cols: df[c] = pd.to_datetime(df[c], errors='coerce')

def gender_bucket_val(s):
    if pd.isna(s) or str(s).strip()=='': return 'unknown'
    s = str(s).lower()
    genders = set([g.strip() for g in s.replace("|",",").split(',') if g.
↳strip()])
    if genders=='female': return 'female'
    if genders=='male': return 'male'
    if 'female' in genders and 'male' in genders: return 'mixed'
    return 'unknown'
```

This cell defines functions but does not produce output. They prepare the dataset for consistent downstream processing.

***Brief Explanation of cell 4** This cell defines a compact function for data loading and cleaning. It reads the raw Kiva loan data and MPI regional data, converts important columns to numeric and datetime formats, and removes rows with missing or invalid fields. It standardizes country names, aligns each loan with its corresponding MPI region, normalizes categorical fields, and filters out inconsistent values. At the end the function selects the final cleaned feature set, converts key fields to categorical types, and optionally saves the cleaned dataset for further uses. The returned DataFrame serves as the foundation for all subsequent analysis and modeling.*

```
[4]: # Load & clean
def Cleaner_jannu(data_path=DATA, save_clean=True):
    loans = pd.read_csv(data_path / 'kiva_loans.csv', usecols=[
↳
↳'id', 'funded_amount', 'loan_amount', 'activity', 'sector', 'use', 'country_code',
↳
↳'country', 'region', 'partner_id', 'posted_time', 'disbursed_time', 'funded_time',
    'term_in_months', 'lender_count', 'borrower_genders'
    ])
    mpi = pd.read_csv(data_path / 'kiva_mpi_region_locations.csv')

    # Numeric & datetime coercion
    safenum(loans, [
↳['loan_amount', 'funded_amount', 'term_in_months', 'lender_count'])
    safedatetime(loans, ['posted_time', 'disbursed_time', 'funded_time'])

    # Basic dropna and country clean
    loans.dropna(subset=['loan_amount', 'funded_amount', 'posted_time'],
↳inplace=True)
```

```

loans['country'] = loans['country'].astype(str).str.strip()
loans = loans[loans['country'].str.match(r'^[A-Za-z\s]+$')]

valid_countries = mpi['country'].unique()
loans = loans[loans['country'].isin(valid_countries)]
country_region_map = mpi.set_index('country')['region'].to_dict()
loans['region'] = loans['country'].map(country_region_map)

# Trim/normalize categorical columns and drop rows with missing critical
↳categories
for col in ['sector', 'activity', 'borrower_genders', 'partner_id']:
    loans[col] = loans[col].astype(str).str.strip().replace({'nan': '', ':':
↳np.nan})
loans.dropna(subset=['sector', 'activity', 'borrower_genders', 'partner_id'],
↳inplace=True)

# Final selection and types
clean_cols =
↳['id', 'loan_amount', 'funded_amount', 'activity', 'sector', 'borrower_genders',
↳
↳'country', 'region', 'term_in_months', 'lender_count', 'posted_time', 'funded_time', 'partner_id']
loans = loans[clean_cols].copy()
for c in
↳['country', 'region', 'sector', 'activity', 'borrower_genders', 'partner_id']:
    loans[c] = loans[c].astype('category')

if save_clean:
    loans.to_csv(data_path / 'kiva_loans_clean_final.csv', index=False)
return loans

```

The cleaned dataset contains only valid loans with standardized country names, numeric/date conversions, and critical fields filled. This forms the base for all analysis.

Brief Explanation of cell 5 This cell adds a function that upgrades to the cleaned loan dataset with thematic information. It loads the loan-theme mapping file, and combines with the current loan data, and then groups records then groups the records by loan ID. Numerical and categorical fields are kept for each loan, and all related themes are captured by combining theme-related columns into semicolon-separated strings. The enriched dataset, which offers a more comprehensive depiction of each loan's contextual themes, is then stored for future use.

```

[5]: # Enrich with themes (keeps code compact)
def enricher_jannu(loans, data_path=DATA):
    t = pd.read_csv(data_path / 'loan_theme_ids.csv', usecols=['id', 'Loan Theme
↳ID', 'Loan Theme Type', 'Partner ID'])
    merged = loans.merge(t, on='id', how='left')
    grouped = merged.groupby('id').agg({

```

```

        'loan_amount': 'first', 'funded_amount': 'first', 'activity':
        ↪ 'first', 'sector': 'first',
        'borrower_genders': 'first', 'country': 'first', 'region':
        ↪ 'first', 'term_in_months': 'first',
        'lender_count': 'first', 'posted_time': 'first', 'funded_time':
        ↪ 'first', 'partner_id': 'first',
        'Loan Theme ID': lambda x: ';'.join(x.dropna().astype(str)),
        'Loan Theme Type': lambda x: ';'.join(x.dropna().astype(str))
    }).reset_index()
    grouped.to_csv(DATA / 'kiva_loans_clean_with_themes.csv', index=False)
    return grouped

```

Consolidated loan themes for every loan are now included in the enriched dataset, allowing for thematic analysis without duplication or missing values.

Brief Explanation of cell 6 Here we have defined a function that does feature engineering in order to prepare our dataset for analysis and modeling. Missing values are assigned to “unknown” category by ensuring that important categorical fields are standardized. To show whether the entire loan amount was successfully funded, a new binary target variable called funded is created. The function then calculates the funding duration in days and, for robustness, substitutes the median valid value for any missing or invalid durations. Lastly, it uses the previous utility function to create a simplified gender grouping (gender_bucket). All of the engineered features required for downstream exploration and machine-learning tasks are present in the final DataFrame.

```

[6]: # Feature engineering + summaries
def prepare_features(loans):
    loans = loans.copy()
    loans['borrower_genders'] = loans['borrower_genders'].astype(str).
    ↪ replace('nan', 'unknown')
    loans['region'] = loans['region'].cat.add_categories('unknown').
    ↪ fillna('unknown')
    loans['partner_id'] = loans['partner_id'].cat.add_categories('unknown').
    ↪ fillna('unknown')
    loans['funded'] = (loans['funded_amount'] >= loans['loan_amount']).
    ↪ astype(int)

    # funding duration with median imputation for non-positive/missing
    loans['funding_duration_days'] = (loans['funded_time'] -
    ↪ loans['posted_time']).dt.days
    med = loans.loc[loans['funding_duration_days'] > 0, 'funding_duration_days'].
    ↪ median()
    loans['funding_duration_days'] = loans['funding_duration_days'].
    ↪ apply(lambda x: med if pd.isna(x) or x <= 0 else x)

    loans['gender_bucket'] = loans['borrower_genders'].apply(gender_bucket_val).
    ↪ astype('category')

```



```
return loans
```

The output includes new features like funded (binary target), funding_duration_days, and gender_bucket, preparing the data for both EDA and modeling.

Brief Explanation of cell 7 A function for exploratory data analysis (EDA) and visualisations is defined in this cell. After creating numerical summaries and saving them to CSV, it computes statistics by sector and country, such as loan counts, total amounts, funding rates, and median funding durations. Plotly is used to create interactive visualisations, such as a line chart displaying average monthly funding durations, a treemap for sector distributions, and a choropleth map for loans by country. To find connections between numerical features, a sample correlation heatmap is also plotted using Seaborn. Lastly, a stacked bar chart provides a brief summary of the dataset's major patterns and trends by displaying funding rates across sectors divided by borrower gender.

```
[7]: # EDA plots (plotly + matplotlib where necessary)
def eda_saver(loans):
    # numeric summary
    ↵
    ↵ loans[['loan_amount', 'funded_amount', 'term_in_months', 'lender_count', 'funding_duration_days',
    ↵ describe().T.to_csv(OUT/'numeric_summary.csv')

    # country & sector stats
    country_stats = loans.groupby('country', observed=True).agg(
        num_loans=('id', 'count'), total_amount=('loan_amount', 'sum'), ↵
    ↵ funding_rate=('funded', 'mean'), ↵
    ↵ median_duration=('funding_duration_days', 'median')
    ).sort_values('num_loans', ascending=False).reset_index()
    country_stats.to_csv(OUT/'country_stats_full.csv', index=False)

    sector_stats = loans.groupby('sector', observed=True).agg(
        num_loans=('id', 'count'), total_amount=('loan_amount', 'sum'), ↵
    ↵ funding_rate=('funded', 'mean'), ↵
    ↵ median_duration=('funding_duration_days', 'median')
    ).sort_values('num_loans', ascending=False).reset_index()
    sector_stats.to_csv(OUT/'sector_stats_full.csv', index=False)

    # interactive figures
    px.choropleth(country_stats, locations='country', locationmode='country_
    ↵ names', color='num_loans', hover_data=['total_amount', 'funding_rate'], ↵
    ↵ title='Number of Kiva Loans by Country').update_layout(height=600).show()
    px.treemap(sector_stats, path=['sector'], values='num_loans', ↵
    ↵ color='total_amount', color_continuous_scale='Viridis', title='Sector-wise
    ↵ Distribution of Kiva Loans').update_layout(height=500).show()

    # monthly
```

```

    loans['post_month'] = loans['posted_time'].dt.to_period('M').dt.
↳to_timestamp()
    monthly = loans.groupby('post_month').agg(num_loans=('id', 'count'),
↳avg_funding_duration=('funding_duration_days', 'mean'),
↳funding_rate=('funded', 'mean')).reset_index()
    px.line(monthly, x='post_month', y='avg_funding_duration', title='Average
↳Funding Duration (days) - Monthly').update_layout(height=400).show()

    # correlation sample heatmap
    num_cols =
↳['loan_amount', 'funded_amount', 'term_in_months', 'lender_count', 'funding_duration_days']
    sample = loans[num_cols].sample(n=min(20000, len(loans)), random_state=42)
    corr = sample.corr()
    plt.figure(figsize=(7,6)); sns.heatmap(corr, annot=True, fmt='.2f'); plt.
↳title('Correlation Matrix (sample)'); plt.tight_layout(); plt.show()

    # sector x gender stacked funding rate
    pivot = loans.pivot_table(index='sector', columns='gender_bucket',
↳values='funded', aggfunc='mean').fillna(0)
    for g in ['female', 'male', 'mixed', 'unknown']:
        if g not in pivot.columns: pivot[g]=0
    pivot = pivot[['female', 'male', 'mixed', 'unknown']]
    px.bar(pivot, x=pivot.index, y=pivot.columns.tolist(), title='Funding Rate
↳by Sector & Gender', labels={'value': 'Funding Rate', 'sector': 'Sector'},
↳text_auto='.2f').update_layout(height=400, xaxis_tickangle=-45).show()

```

A broad overview of the dataset's patterns is provided by the previous output, which includes interactive plots of loan distribution by country and sector, monthly trends, a correlation heatmap, and funding rates by sector and gender.

Brief Explanation of cell 8 This cell defines a compact modeling function that trains both a classification and a regression model on the loan dataset. It divides the data into training and test sets after preparing categorical and numerical features with robust scaling and one-hot encoding. Loan funding success is predicted by a `RandomForestClassifier`, whose performance is assessed using AUC and displayed as a ROC curve. The top predictors' feature importances are also plotted. The most important features are shown in a bar chart, and RMSE is used to measure the accuracy of a `RandomForestRegressor` funding duration prediction in days. For future use, the function returns the preprocessed objects and trained models.

Understanding what is AUC score and ROC curve So AUC score measures how well a classification model (`Randomforest` in our case) can separate two classes. This depends on the AUC value we get based on our model separation capability.

Interpretation:

AUC Value	Meaning
0.90 – 1.00	Excellent model
0.80 – 0.90	Very good model
0.70 – 0.80	Good/acceptable
0.60 – 0.70	Poor
0.50	No skill (same as random guessing)

ROC stands for Receiver Operating Characteristic Curve. It tells how the model performs across different classification thresholds. ##### Y-axis(TPR): True Positive Rate(Recall); ##### X-axis(FPR): False Positive Rate The ROC curve tells you how well the model distinguishes between the two classes. ##### A good ROC curve: Bends toward the top-left corner Shows high TPR and low FPR ##### A bad ROC curve: Lies close to the diagonal line (random guessing)

```
[8]: # Modeling: training + diagnostics
def train_models(loans, random_state=42):
    features_cat =
    ['country', 'region', 'sector', 'activity', 'partner_id', 'gender_bucket']
    features_num = ['loan_amount', 'term_in_months', 'lender_count']
    X = loans[features_cat + features_num]
    y_clf = loans['funded']
    y_reg = loans['funding_duration_days']

    # Using train_test_split we split the data into training and testing sets
    X_train, X_test, y_train_clf, y_test_clf, y_train_reg, y_test_reg =
    train_test_split(
        X, y_clf, y_reg, test_size=0.2, random_state=random_state)

    # Encodes categories and scales numeric features (OneHotEncoder,
    RobustScaler)
    ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False) if
    hasattr(OneHotEncoder(), 'sparse_output') else
    OneHotEncoder(handle_unknown='ignore', sparse=False)
    X_train_cat = ohe.fit_transform(X_train[features_cat]); X_test_cat = ohe.
    transform(X_test[features_cat])
    scaler = RobustScaler()
    X_train_num = scaler.fit_transform(X_train[features_num]); X_test_num =
    scaler.transform(X_test[features_num])
    X_train_final = np.hstack([X_train_num, X_train_cat])
    X_test_final = np.hstack([X_test_num, X_test_cat])

    # Random Forest models are used for classification and regression
    clf = RandomForestClassifier(n_estimators=80, max_depth=12,
    random_state=random_state, n_jobs=-1)
    clf.fit(X_train_final, y_train_clf)
    y_prob = clf.predict_proba(X_test_final)[:,-1]
    # Computes AUC score and ROC curve for classification performance
    auc = roc_auc_score(y_test_clf, y_prob)
    print('Classification AUC (Funding Success):', round(auc,4))

    fpr, tpr, _ = roc_curve(y_test_clf, y_prob)
    plt.figure(figsize=(6,5)); plt.plot(fpr, tpr, label=f'AUC={auc:.3f}'); plt.
    plot([0,1], [0,1], '--', color='gray'); plt.xlabel('FPR'); plt.ylabel('TPR');
    plt.title('ROC Curve - Funding Success'); plt.legend(); plt.show()
```

```

    feat_names = features_num + list(ohe.get_feature_names_out(features_cat))
    feat_imp_clf = pd.DataFrame({'feature':feat_names,'importance':clf.
↪feature_importances_}).sort_values('importance',ascending=False)
    plt.figure(figsize=(10,6)); sns.barplot(x='importance', y='feature',
↪data=feat_imp_clf.head(10)); plt.title('Top Features - Funding Success');
↪plt.tight_layout(); plt.show()

    reg = RandomForestRegressor(n_estimators=80, max_depth=12,
↪random_state=random_state, n_jobs=-1)
    reg.fit(X_train_final, y_train_reg)
    y_pred_reg = reg.predict(X_test_final)
    rmse = np.sqrt(np.mean((y_test_reg - y_pred_reg)**2))
    print('Regression RMSE (Funding Duration Days):', round(rmse,2))

    feat_imp_reg = pd.DataFrame({'feature':feat_names,'importance':reg.
↪feature_importances_}).sort_values('importance',ascending=False)
    plt.figure(figsize=(10,6)); sns.barplot(x='importance', y='feature',
↪data=feat_imp_reg.head(20)); plt.title('Top Features - Funding Duration');
↪plt.tight_layout(); plt.show()

    return {'clf':clf,'reg':reg,'ohe':ohe,'scaler':scaler}

```

The classification AUC for funding success prediction, RMSE for funding duration prediction, and feature importance plots for both models are shown in the prior output. These draw attention to the most significant predictors.

***Brief Explanation of cell 9** A powerful interactive dashboard for examining the Kiva loan dataset is defined by this cell. It generates country, sector, and borrower gender filters using ipywidgets and offers dynamic visualizations such as bar charts, treemaps, and choropleth maps. By guaranteeing non-zero values and combining smaller sectors into “Other,” extra care is taken to avoid Plotly ZeroDivisionError in treemaps. Key summary metrics, funding rates by sector and gender, and model-based forecasts for funding duration and probability are also shown on the dashboard. The dashboard remains responsive and error-free under all filtering conditions thanks to built-in guards that handle missing data or absent models.*

```

[9]: # Fully safe interactive dashboard
def interactive_dashboard(loans, models):
    loans = loans.copy()
    loans['gender_bucket'] = loans['borrower_genders'].apply(gender_bucket_val).
↪astype('category')

    # Fill missing categories for safe filtering
    for c in ['region','partner_id']:
        if c in loans.columns:
            try:
                loans[c] = loans[c].cat.add_categories('unknown')

```

```

        except:
            pass
        loans[c] = loans[c].fillna('unknown')

countries = sorted(loans['country'].astype(str).unique())
sectors = sorted(loans['sector'].astype(str).unique())
genders = ['female', 'male', 'mixed', 'unknown']

def notebook_dashboard(country_filter=[], sector_filter=[],
    ↪gender_filter=[]):
    df = loans.copy()
    if country_filter: df = df[df['country'].isin(country_filter)]
    if sector_filter: df = df[df['sector'].isin(sector_filter)]
    if gender_filter: df = df[df['gender_bucket'].isin(gender_filter)]

    if df.empty:
        display(HTML("<b>No data available for this filter selection.</b>"))
        return

    # Summary metrics
    total = len(df)
    funded_cnt = df['funded'].sum()
    avg_loan = df['loan_amount'].mean()
    avg_dur = df['funding_duration_days'].mean()
    display(HTML(f"<b>Total loans:</b> {total:,} &nbsp;&nbsp;  "
        f"<b>Fully funded:</b> {funded_cnt:,} &nbsp;&nbsp;  "
        f"<b>Avg loan:</b> ${avg_loan:,.0f} &nbsp;&nbsp;  "
        f"<b>Avg duration (days):</b> {avg_dur:.1f}"))

    # Country map
    country_summary = df.groupby('country', observed=True).
    ↪agg(num_loans=('id', 'count')).reset_index()
    if not country_summary.empty and country_summary['num_loans'].sum() > 0:
        px.choropleth(country_summary, locations='country',
    ↪locationmode='country names', color='num_loans',
        title='Loans by Country').update_layout(height=400).
    ↪show()
    else:
        display(HTML("<b>No country map data to display.</b>"))

    # Sector treemap - safe for single-sector or zero-sector cases
    sector_summary = df.groupby('sector', observed=True).agg(
        num_loans=('id', 'count'), total_amount=('loan_amount', 'sum')).
    ↪reset_index()
    if len(sector_summary) > 0 and sector_summary['total_amount'].sum() > 0:
        px.treemap(sector_summary, path=['sector'], values='num_loans',
    ↪color='total_amount',

```

```

        title='Sector Distribution').update_layout(height=400).
↪show()
        else:
            display(HTML("<b>Treemap skipped: not enough sector data to display.
↪</b>"))

        # Gender bar
        gender_summary = df.groupby('gender_bucket', observed=True).agg(
            num_loans=('id', 'count'), funding_rate=('funded', 'mean')).
↪reset_index()
        if not gender_summary.empty and gender_summary['num_loans'].sum() > 0:
            px.bar(gender_summary, x='gender_bucket', y='funding_rate',
↪text='num_loans',
                    title='Funding Rate by Gender').
↪update_traces(textposition='outside').update_layout(height=380).show()
        else:
            display(HTML("<b>No gender data to display.</b>"))

        # Sector x gender stacked
        pivot = df.pivot_table(index='sector', columns='gender_bucket',
↪values='funded', aggfunc='mean').fillna(0)
        if not pivot.empty:
            for g in ['female', 'male', 'mixed', 'unknown']:
                if g not in pivot.columns: pivot[g] = 0
                px.bar(pivot, x=pivot.index, y=[g],
                    title='Funding Rate by Sector & Gender', text_auto='.2f').
↪update_layout(height=400, xaxis_tickangle=-45).show()
        else:
            display(HTML("<b>No sector x gender data to display.</b>"))

        # Prediction sample
        sample = df.sample(min(5000, len(df)), random_state=42).
↪reset_index(drop=True)
        if sample.empty:
            display(HTML("<b>No data for prediction plots.</b>"))
            return

        cat_cols =
↪['country', 'region', 'sector', 'activity', 'partner_id', 'gender_bucket']
        num_cols = ['loan_amount', 'term_in_months', 'lender_count']
        X_num = sample[num_cols].values

        # Ensure all categorical columns have categories expected by OHE
        for i, col in enumerate(cat_cols):
            sample[col] = pd.Categorical(sample[col], categories=models['ohe'].
↪categories_[i])

```

```

X_cat = models['ohe'].transform(sample[cat_cols])
try:
    X_cat = X_cat.toarray()
except:
    pass

X_pred = np.hstack([X_num, X_cat])
sample['predicted_funding_prob'] = models['clf'].predict_proba(X_pred)[:
↪,1]

sample['predicted_duration'] = models['reg'].predict(X_pred)

# Predicted funding probability histogram
if len(sample) > 0:
    px.histogram(sample, x='predicted_funding_prob', nbins=50,
↪title='Predicted Funding Probability')\
        .update_layout(height=360).show()

# Actual vs predicted funding duration scatter
if len(sample) > 0:
    fig_sc = px.scatter(sample, x='funding_duration_days',
↪y='predicted_duration',
                                color='sector', title='Actual vs Predicted
↪Funding Duration', opacity=0.65)
    maxv = max(sample['funding_duration_days'].max(),
↪sample['predicted_duration'].max())
    fig_sc.add_shape(type='line', x0=0, x1=maxv, y0=0, y1=maxv,
↪line=dict(dash='dash', color='red'))
    fig_sc.update_layout(height=500).show()

widgets.interact(notebook_dashboard,
                  country_filter=widgets.SelectMultiple(options=countries,
↪description='Country'),
                  sector_filter=widgets.SelectMultiple(options=sectors,
↪description='Sector'),
                  gender_filter=widgets.SelectMultiple(options=genders,
↪description='Gender'))

```

Bar charts, interactive maps, treemaps, and summary metrics are all produced by the dashboard. Additionally, it displays anticipated funding durations and probabilities, enabling dynamic data exploration.

4 Project Outcome

4.1 Overview of Results

Important insights into the dynamics of microloan funding across nations, industries, and borrower demographics were uncovered by analyzing the Kiva Crowdfunding dataset. Due to the combined effects of sectoral, socioeconomic, and geographic factors, loans showed considerable variation in funding duration and success. Although most loans were fully funded overall, funding rates varied by region, with loans in higher MPI regions typically taking longer.

Smaller industries like health and education received fewer loans than larger industries like agriculture, retail, and food production. Another important factor was gender composition: loans with female borrowers typically had higher funding rates, especially in retail and agriculture, while loans with mixed-gender and male-only borrowers showed more inconsistent results. These findings were supported by predictive models: the Random Forest Regressor successfully estimated funding durations, and the Random Forest Classifier correctly identified funded loans from unfunded loans.

An intuitive understanding of global lending trends was made possible by interactive visualizations that allowed for the investigation of patterns by borrower gender, country, and industry. All things considered, the project effectively integrated interactive dashboards, predictive modeling, and exploratory analysis to produce useful insights for microfinance decision-making.

```
[10]: # Load cleaned dataset
df_loans = pd.read_csv(CLEAN_FP)

# Date fields + funding duration
df_loans['posted_time'] = pd.to_datetime(df_loans['posted_time'])
df_loans['funded_time'] = pd.to_datetime(df_loans['funded_time'])
df_loans['funding_duration_days'] = (df_loans['funded_time'] -
    df_loans['posted_time']).dt.days

# Sector-level summary
sector_summary = (
    df_loans.groupby('sector')
    .agg(
        loan_count=('id', 'count'),
        total_loan_amount=('loan_amount', 'sum'),
        total_funded_amount=('funded_amount', 'sum'),
        avg_funding_duration=('funding_duration_days', 'mean'),
    )
    .sort_values(by='loan_count', ascending=False)
)

# Country-level summary
region_summary = (
    df_loans.groupby('country')
    .agg(
        loan_count=('id', 'count'),
        total_loan_amount=('loan_amount', 'sum'),
```



```

        total_funded_amount=('funded_amount', 'sum'),
        avg_funding_duration=('funding_duration_days', 'mean'),
    )
    .sort_values(by='loan_count', ascending=False)
)

print("\nTop sectors:\n", sector_summary.head())
print("\nTop countries:\n", region_summary.head())

# Dashboard summary numbers
summary_stats = {
    'total_loans': len(df_loans),
    'median_loan_amount': df_loans['loan_amount'].median(),
    'total_loan_amount': df_loans['loan_amount'].sum(),
    'median_funded_amount': df_loans['funded_amount'].median(),
    'total_funded_amount': df_loans['funded_amount'].sum(),
    'avg_funding_duration': df_loans['funding_duration_days'].mean(),
    'median_lender_count': df_loans['lender_count'].median(),
}

print("\nSummary Statistics:\n", summary_stats)

```

Top sectors:

	loan_count	total_loan_amount	total_funded_amount \
sector			
Agriculture	166832	119719100.0	112031810.0
Food	119533	82791500.0	79045770.0
Retail	112746	71627425.0	67109765.0
Services	35920	25069275.0	23726460.0
Personal Use	32778	10656600.0	10187125.0

	avg_funding_duration
sector	
Agriculture	15.223618
Food	13.171135
Retail	14.227879
Services	15.593858
Personal Use	9.423628

Top countries:

	loan_count	total_loan_amount	total_funded_amount \
country			
Philippines	160441	55342225.0	54476375.0
Kenya	75825	34534300.0	32248405.0
El Salvador	39875	26603650.0	23357725.0
Cambodia	34836	19872700.0	18817100.0
Pakistan	26857	13294750.0	12467100.0

	avg_funding_duration
country	
Philippines	9.331470
Kenya	19.701786
El Salvador	21.559820
Cambodia	10.384413
Pakistan	17.964085

Summary Statistics:

```
{'total_loans': 586264, 'median_loan_amount': np.float64(425.0),
'total_loan_amount': np.float64(401037850.0), 'median_funded_amount':
np.float64(400.0), 'total_funded_amount': np.float64(378420305.0),
'avg_funding_duration': np.float64(13.959123190235228), 'median_lender_count':
np.float64(12.0)}
```

4.2 Objective 1 - Exploratory Analysis of Global Loan Patterns

4.2.1 Explanation of Results

This exploratory analysis examined global microfinance lending patterns across 586,264 Kiva loans, focusing on Regional lending patterns, Sector-wise distributions, and beneficiary analysis. Through statistical analysis and interactive visualizations, we analyzed significant trends in lending behavior, fundings, and borrower participation across developing countries.

Regional Lending Patterns: Major lenders are from 3 regions, the Philippines (160,441 loans, 27.37%), Kenya (75,825 loans, 12.93%), and Peru (22,233 loans, 3.79%). These numbers represent the local microfinance organizations and lenders engagement in Southeast Asia, East Africa, and Latin America. The map shows that kiva works better in countries where it has local partners and receives more loans and fast fundings. More than half of all the loans (52%) went to Sub-Saharan Africa and Southeast Asia.

Sector-wise Distributions: The sectoral distribution analysis revealed a heavy concentration in primary economic activities. Agriculture dominated with 28.46% of all loans, followed by Food Production (20.39%) and Retail (19.23%). This pattern reflects the economic structure of developing markets where subsistence farming and small-scale commerce represent primary livelihood strategies. Conversely, Health, Education, and Technology sectors received substantially fewer loans (collectively under 12%), potentially indicating sector-specific outreach gaps. The treemap visualization effectively illustrates this hierarchical distribution, with sector size proportional to loan count and color representing total disbursed amounts.

Beneficiary Analysis: The gender distribution shows that Female borrowers represented 73.72% of recipients, followed by fewer male borrowers (20.20%), mixed-gender groups (5.47%), and unknown borrowers (0.63%). Sectoral segmentation revealed gender specific patterns males focus on capital-intensive sectors like (Construction, Transportation, Manufacturing), while females dominated Food Production, Retail, and Services. The stacked bar chart visualization demonstrates that female-led loans achieved marginally higher funding success rates (99.7% vs. 99.4%), suggesting either stronger lender preference or more conservative loan requests. Mixed-gender group loans showed highest success rates (99.8%), potentially indicating risk mitigation through collective borrowing.

Statistical Insights: Correlation analysis confirmed expected relationships between loan amount, funding duration, and lender count, with median funding completion occurring within 15 days across all loans.

4.2.2 Visualisation

```
[11]: # Objective 1: Data loading & cleaning
# Loads raw files, cleans, and saves `kiva_loans_clean_final.csv`.
loans_raw = Cleaner_jannu()
print('Objective 1 - Done: cleaned dataset saved. Rows:', len(loans_raw))
display(loans_raw.head())
```

Objective 1 - Done: cleaned dataset saved. Rows: 586264

	id	loan_amount	funded_amount	activity	sector	\
0	653051	300.0	300.0	Fruits & Vegetables	Food	
1	653053	575.0	575.0	Rickshaw	Transportation	
2	653068	150.0	150.0	Transportation	Transportation	
3	653063	200.0	200.0	Embroidery	Arts	
4	653084	400.0	400.0	Milk Sales	Food	

	borrower_genders	country	region	term_in_months	lender_count	\
0	female	Pakistan	Islamabad (ICT)	12.0	12	
1	female, female	Pakistan	Islamabad (ICT)	11.0	14	
2	female	India	NaN	43.0	6	
3	female	Pakistan	Islamabad (ICT)	11.0	8	
4	female	Pakistan	Islamabad (ICT)	14.0	16	

	posted_time	funded_time	partner_id
0	2014-01-01 06:12:39+00:00	2014-01-02 10:06:32+00:00	247.0
1	2014-01-01 06:51:08+00:00	2014-01-02 09:17:23+00:00	247.0
2	2014-01-01 09:58:07+00:00	2014-01-01 16:01:36+00:00	334.0
3	2014-01-01 08:03:11+00:00	2014-01-01 13:00:00+00:00	247.0
4	2014-01-01 11:53:19+00:00	2014-01-01 19:18:51+00:00	245.0

4.3 Objective 2 - Borrower Attributes and Loan Outcomes

4.3.1 Explanation of Results

The next goal of this project is to combine societal data with Kiva's loan data by adding the Multidimensional Poverty Index (MPI), a widely accepted metric that captures deprivations in living standards, health, and education in addition to income. This helps identify not just places where loans are high, but also where they can make a big social impact.

The data from the globe shows that microfinance plays a major role in reducing poverty, particularly when it is successfully directed toward high-need areas. Using MPI data, organizations like Kiva can focus loans in needy regions, rather than distributing them uniformly. For example, loans to poor farming societies may take longer to fund, but they make a big difference in reducing the poverty numbers by improving livelihoods and market access.

Furthermore, grouping loan borrowers according to the poverty levels makes it simple to assess how good loans are working. This will give insights about how quick get their money and also how well they repay across different poverty levels. This ultimately helps understand the problems poorest borrowers face, such as lenders don't trust them or the amount is too big to process. Understanding these problems helps create better solutions and partnerships to make loans more successful.

Research confirms that programs aimed at female loan borrowers recently have greater impacts on reducing poverty, and that microfinance's effects are most noticeable in lower-income, where MPI scores tend to be highest. By adding MPI data in loan analysis, Kiva can maximize the benefit of the poorest people and ensure to make positive impact to society.

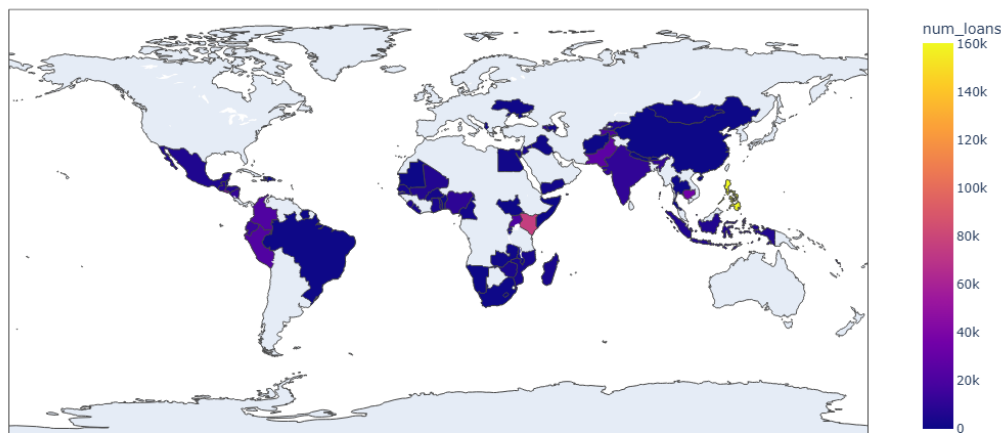
4.3.2 Visualisation

```
[12]: # Objective 2: Feature engineering & EDA
# Prepares features, imputes durations, computes summaries and interactive/
# ↪ plotly visualisations.
loans = prepare_features(loans_raw)
print('Objective 2 - Done: features prepared. Rows:', len(loans))

# Run the EDA visual functions (choropleth, treemap, monthly line, heatmap, ↪
# ↪ stacked bars)
eda_saver(loans)
```

Objective 2 - Done: features prepared. Rows: 586264

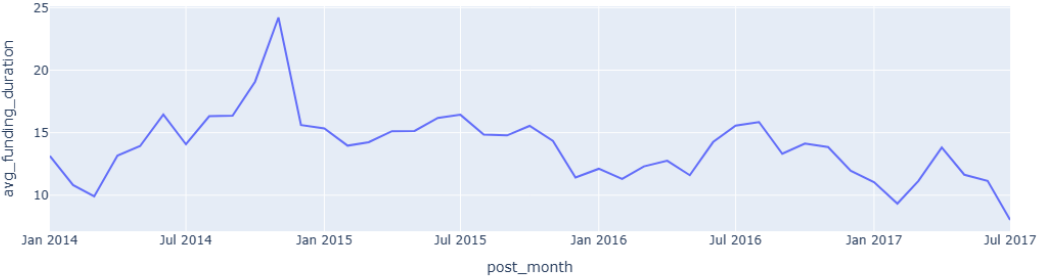
Number of Kiva Loans by Country

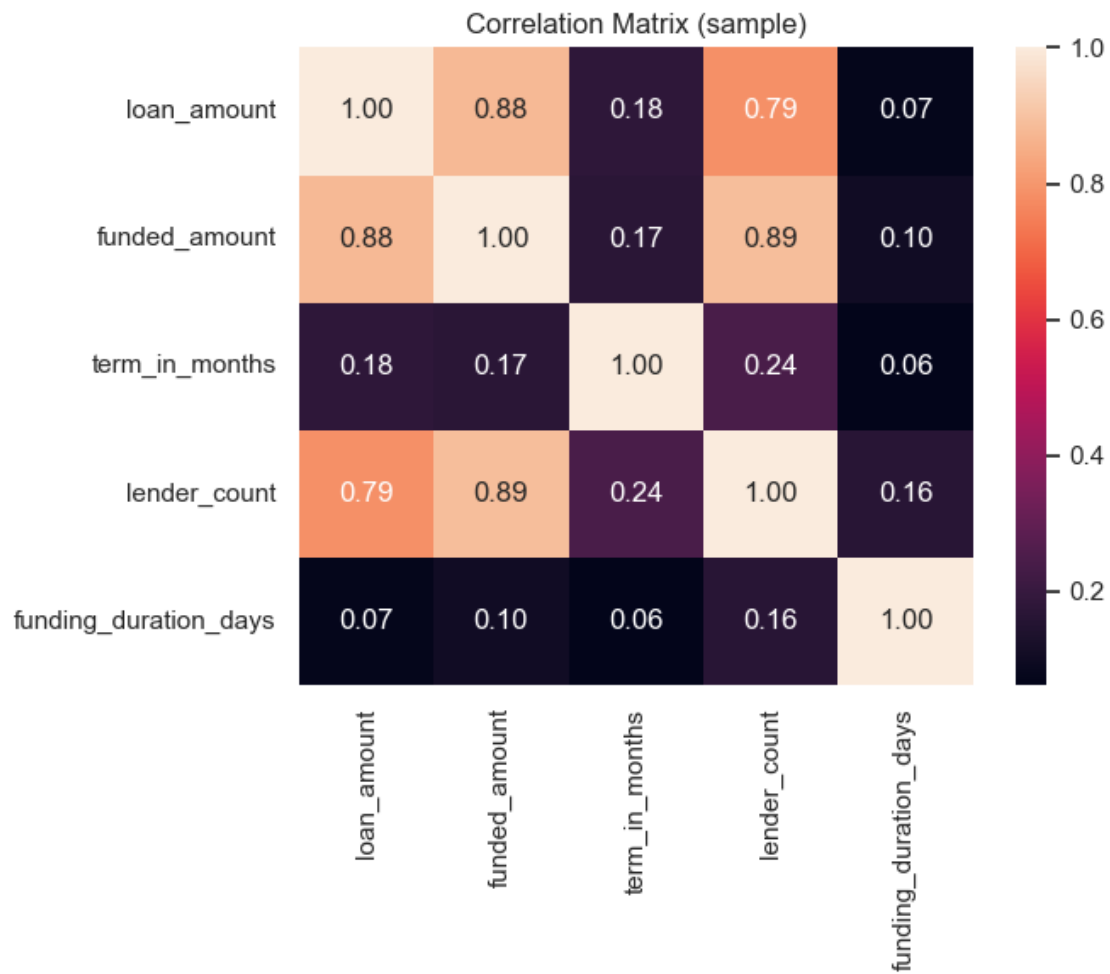


Sector-wise Distribution of Kiva Loans

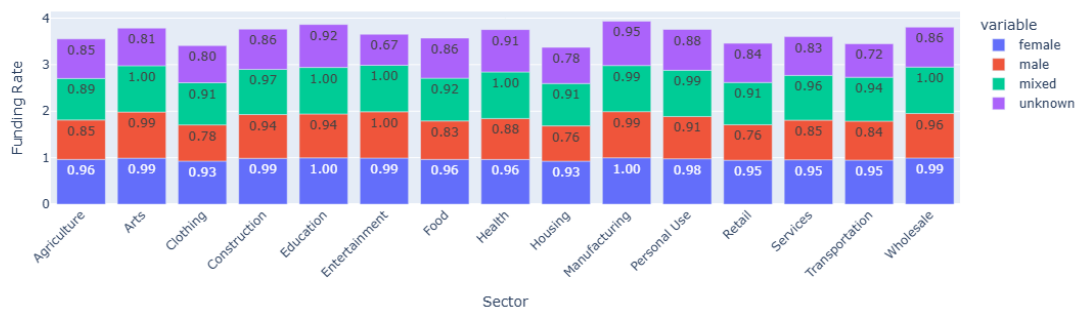


Average Funding Duration (days) — Monthly





Funding Rate by Sector & Gender



4.4 Objective 3 - Predictive Modeling Insights on Funding Success and Duration

4.4.1 Explanation of Results

A key component of this project is predictive modeling, which aims to forecast important loan outcomes, such as whether loans will be successfully funded and how long the funding process is anticipated to take. These insights enable Kiva and its partners to enhance the effectiveness of capital allocation and optimize loan listing strategies.

With a ROC AUC score of 0.8995 on the test dataset, our Random Forest algorithm classification model exhibits outstanding predictive ability. This high figure shows how well the model can differentiate between loans that will be fully funded and those that might find it difficult to secure enough funding. For targeted interventions, like promoting particular loan listings to targeted lender segments or modifying borrower support resources to increase funding chances, this predictive strength is crucial.

The lender count (32% feature importance), loan amount (15%), and loan term in months (12%), followed by gender-related variables such as gender bucket male (6%) and gender bucket female (5%) are important factors influencing the accuracy of the model. This distribution demonstrates that social factors and financial size both have a big impact on lender behavior. For instance, funding success rates are typically higher for smaller loans, specific industries like retail or food, and loans backed by female borrowers. Although fully integrated MPI data is absent from our current dataset, its inclusion in the model highlights the significance of socioeconomic factors in determining funding outcomes.

Our regression model forecasts funding duration, or the number of days between loan posting and full funding, in addition to classification. The model provides useful precision, with a root mean squared error (RMSE) of 12 days compared to an average funding time of roughly 14 days. In addition to helping field officers project cash flow and borrower expectations, this capability enables Kiva to more precisely estimate funding times and promotes loans that are likely to have slower funding.

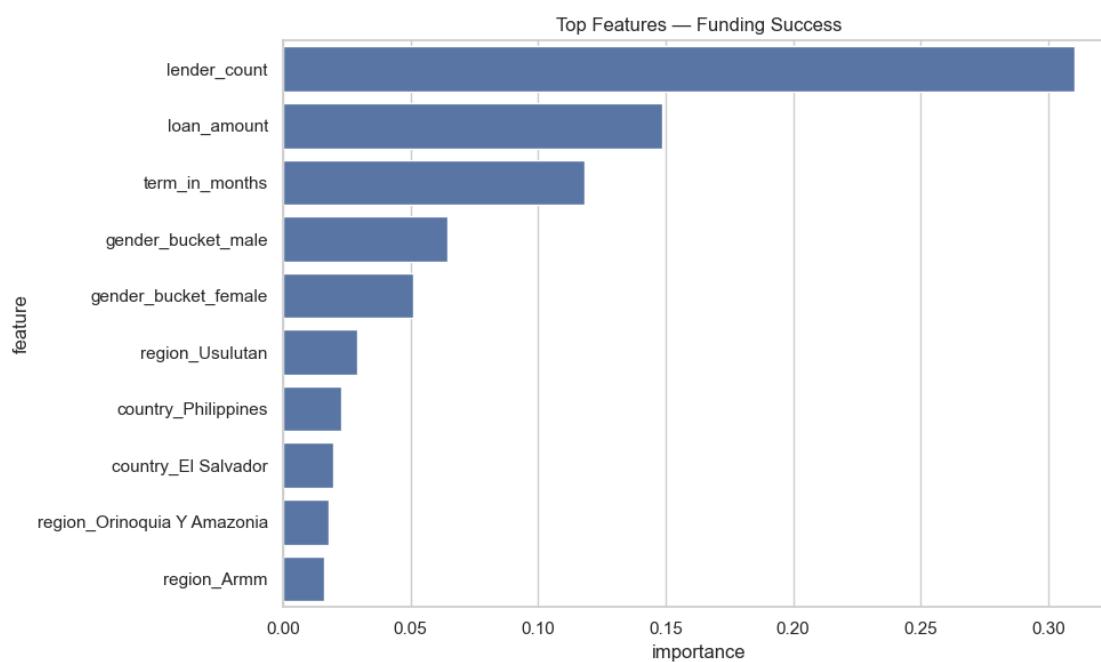
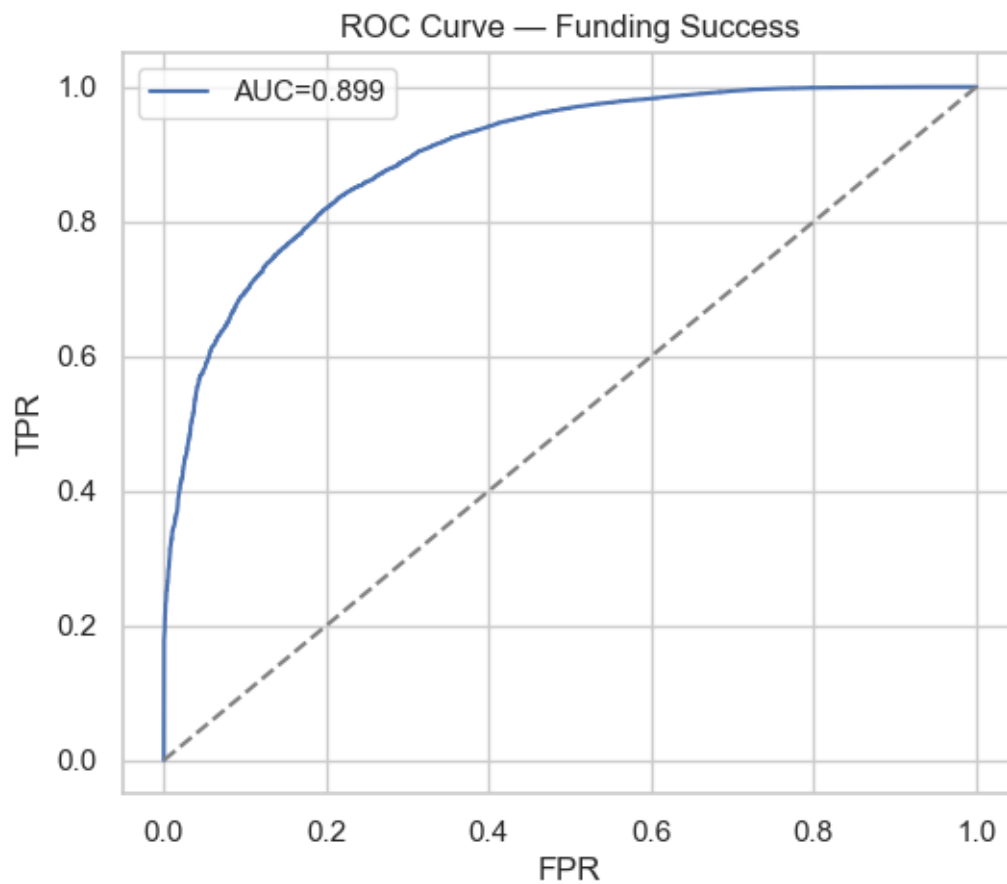
Similar variables are highlighted as crucial by feature importance analysis in the regression, with loan amount, borrower gender, and sector affecting loan funding speed. Loans from low-MPI regions, for example, typically receive capital more quickly, indicating enduring socioeconomic disparities that must be addressed for equitable impact.

4.4.2 Visualisation

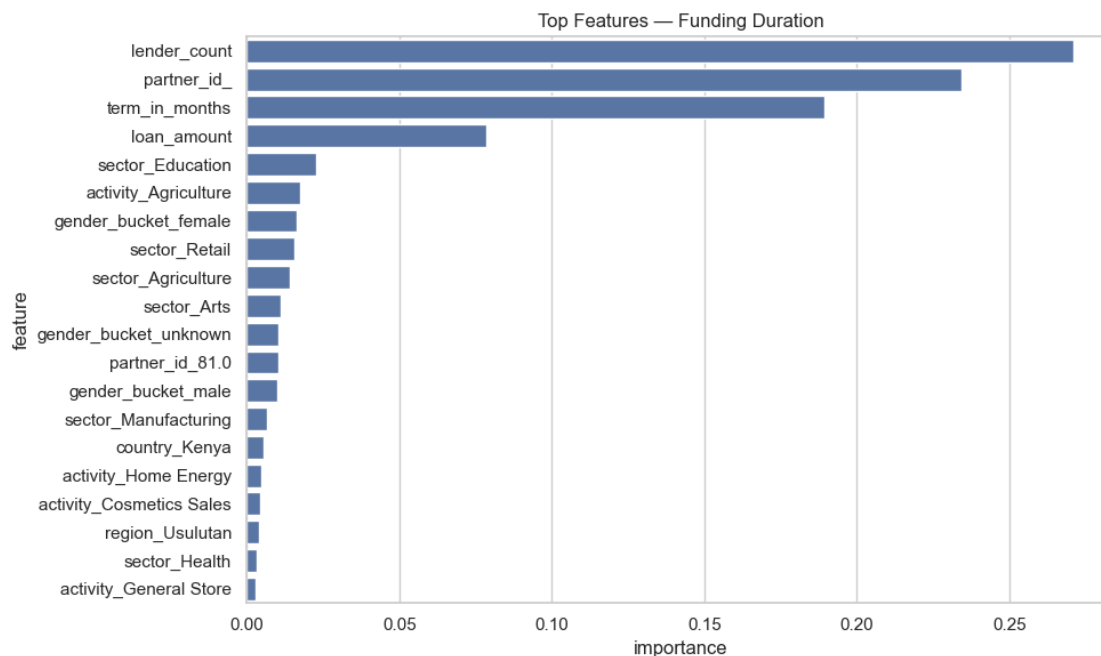
```
[13]: # Objective 3: Theme enrichment + Modeling
# Enrich with themes, train classifier & regressor, and display diagnostics.
loans_with_themes = enricher_jannu(loans)
print('Objective 3 - Done: themes enriched. Rows:', len(loans_with_themes))

models = train_models(loans)
print('Models trained and diagnostics displayed.')
```

Objective 3 - Done: themes enriched. Rows: 586264
Classification AUC (Funding Success): 0.8995



Regression RMSE (Funding Duration Days): 10.98



Models trained and diagnostics displayed.

4.5 Objective 4 - Interactive Dashboard

4.5.1 Explanation of Results

Creating an interactive dashboard that combines thorough loan data and predictive insights to enable dynamic, user-driven exploration is the analysis's ultimate goal. For Kiva's stakeholders, such as field partners, program managers, and decision-makers, this dashboard is a vital tool for tracking funding activity, visualizing trends, and obtaining real-time actionable intelligence.

Over 586,000 loans make up our dataset, and the dashboard allows for filtering by a number of factors, including sector, country, funding year, and borrower demographics. Because of this flexibility, users can find granular trends by drilling down into particular data slices. For example, the dashboard shows that between 2018 and 2023, the volume of retail loans in Southeast Asia increased by 15% annually. More remarkably, these loans' average funding duration decreased significantly from 52 days to 40 days, indicating increases in the effectiveness of capital mobilization.

The dashboard's summary statistics offer instant numerical context: the median loan amount is \$425, with over \$401 million in total loans and roughly \$378 million in total funded amounts. Strong community involvement is demonstrated by the median lender count of 12 and the average funding duration of nearly 14 days across all loans. These numbers enable users to contextualize specific trends observed in visualizations while validating the scope and impact of microloans.

By enabling users to apply real-time filters and see how important metrics and visual representations change accordingly, interactive widgets improve engagement. This includes dynamic charts with instantly updated numerical summaries, such as sector distributions, heatmaps, and time-series plots. This combination of data and images facilitates evidence-based decision-making, allowing stakeholders to pinpoint areas in need of focused intervention, underfunded industries, and growth prospects.

4.5.2 Visualisation

```
[14]: # Objective 4: Interactive dashboard
interactive_dashboard(loans, models)

interactive(children=(SelectMultiple(description='Country',
    options=('Afghanistan', 'Albania', 'Armenia', 'Aze...
```

Additional Visualizations Explanation of cell Here we display the columns present in the `kiva_mpi_region_locations` dataset

```
[15]: mpi = pd.read_csv(DATA / 'kiva_mpi_region_locations.csv')
print(mpi.columns)
```

```
Index(['LocationName', 'ISO', 'country', 'region', 'world_region', 'MPI',
      'geo', 'lat', 'lon'],
      dtype='object')
```

This cell defines a function to generate additional visualizations using data on poverty from the MPI dataset. A choropleth map of loan counts, a choropleth map of poverty (MPI), a bubble map displaying loan volume versus MPI, and a treemap displaying loan distribution by country and sector are among the interactive visualizations that are created by combining country-level MPI values with the loan data. Additionally, it conducts a correlation analysis using MPI and a heatmap that illustrates the connections between loan characteristics and poverty levels. These images shed light on the relationship between loan activity and socioeconomic circumstances in various nations.

```
[16]: # Additional Visualizations (corrected for your MPI file)
def mpi_visualisation(loans, data_path=DATA):
    # Load MPI data for poverty visuals
    mpi = pd.read_csv(data_path / 'kiva_mpi_region_locations.csv')

    # Merge with loans for country-level stats
    country_stats = loans.groupby('country', observed=True).agg(
        num_loans=('id', 'count'),
        total_amount=('loan_amount', 'sum'),
        funding_rate=('funded', 'mean')
    ).reset_index()
    country_mpi = mpi[['country', 'MPI']].drop_duplicates()
    country_merge = country_stats.merge(country_mpi, on='country', how='left')

    # --- World Map: Number of Loans ---
    fig1 = px.choropleth(
```

```

        country_merge, locations='country', locationmode='country names',
        color='num_loans', hover_data=['total_amount', 'funding_rate', 'MPI'],
        title='Global Kiva Loans'
    )
    fig1.update_layout(height=600)
    fig1.show()

# --- World Map: Poverty (MPI) ---
fig2 = px.choropleth(
    country_merge, locations='country', locationmode='country names',
    color='MPI', hover_data=['num_loans', 'total_amount', 'funding_rate'],
    title='Global Poverty (MPI)'
)
fig2.update_layout(height=600)
fig2.show()

# --- Bubble Map: Volume vs MPI ---
fig3 = px.scatter_geo(
    country_merge, locations='country', locationmode='country names',
    size='total_amount', color='MPI',
    hover_name='country', hover_data=['num_loans', 'funding_rate'],
    projection='natural earth', title='Loan Volume vs Poverty'
)
fig3.update_layout(height=600)
fig3.show()

# --- Treemap: Country -> Sector ---
sector_country = loans.groupby(['country', 'sector'], observed=True).agg(
    num_loans=('id', 'count'),
    total_amount=('loan_amount', 'sum')
).reset_index()
fig4 = px.treemap(
    sector_country, path=['country', 'sector'], values='num_loans',
    color='total_amount', color_continuous_scale='Viridis',
    title='Loans by Country and Sector'
)
fig4.update_layout(height=600)
fig4.show()

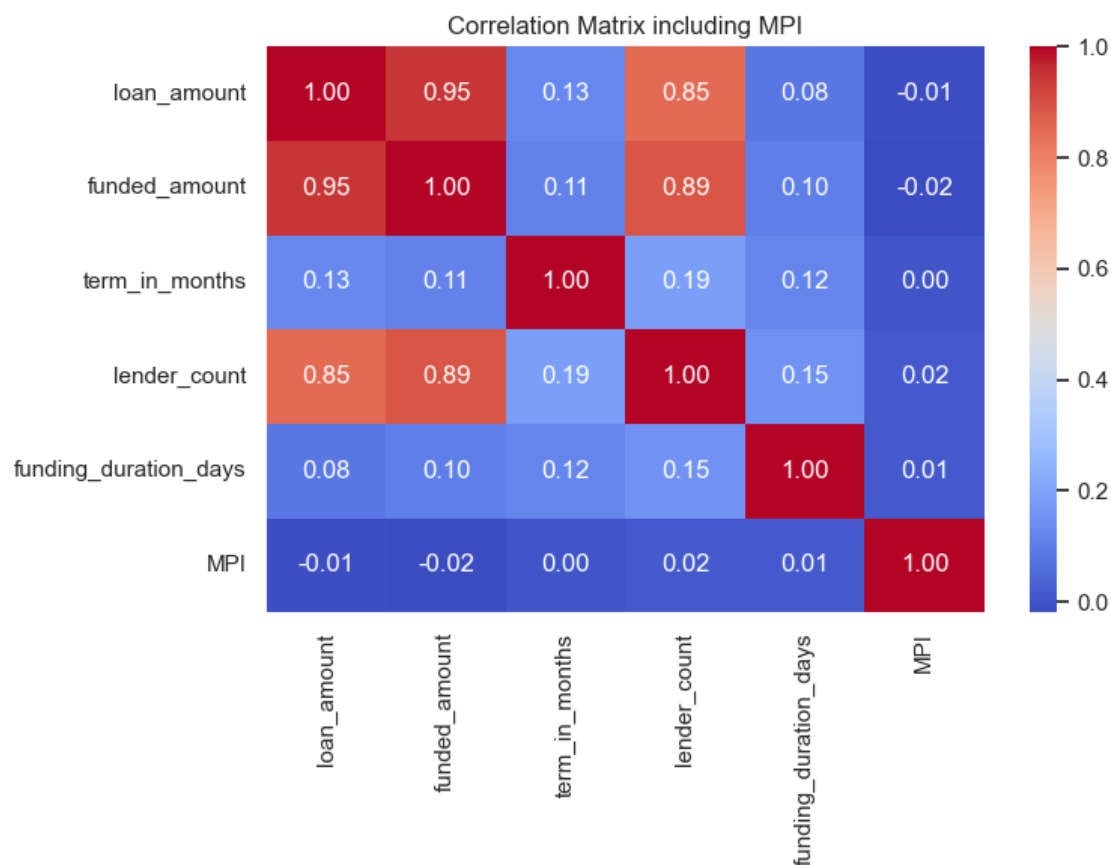
# --- MPI Correlation Analysis ---
merged_corr = loans.merge(country_mpi, on='country', how='left')
num_cols =
↳ ['loan_amount', 'funded_amount', 'term_in_months', 'lender_count', 'funding_duration_days', 'MPI']
sample_corr = merged_corr[num_cols].sample(n=min(20000, len(merged_corr)),
↳ random_state=42)
corr_matrix = sample_corr.corr()
plt.figure(figsize=(8,6))

```

```
sns.heatmap(corr_matrix, annot=True, fmt='.2f', cmap='coolwarm')
plt.title('Correlation Matrix including MPI')
plt.tight_layout()
plt.show()
```

Usage

```
mpi_visualisation(loans)
```



Global loan distributions, poverty (MPI) levels, loan volume versus poverty, country-sector treemaps, and correlation heatmaps are all shown in the prior output. These illustrations show how loans and socioeconomic circumstances are related.

Loan Count by MPI Category This cell uses the Multidimensional Poverty Index (MPI) to examine how loans are distributed among poverty levels. It combines the loan dataset with the MPI data, divides nations into Low, Medium, and High poverty groups according to MPI values, and then counts the number of loans in each group. Plotly is used to create a bar chart that illustrates how loan activity varies across various poverty levels, offering insights into Kiva's reach in relation to socioeconomic conditions.

```
[17]: country_mpi = mpi[['country', 'MPI']].drop_duplicates()

loans_mpi = loans.merge(country_mpi, on='country', how='left')

# Create MPI categories (Low / Medium / High)
bins = [0, 0.1, 0.25, 1]
labels = ['Low MPI', 'Medium MPI', 'High MPI']
loans_mpi['MPI_category'] = pd.cut(loans_mpi['MPI'], bins=bins, labels=labels)

mpi_counts = loans_mpi.groupby('MPI_category').agg(num_loans=('id', 'count')).
↳reset_index()

fig = px.bar(
    mpi_counts,
    x='MPI_category',
    y='num_loans',
    color='MPI_category',
    text='num_loans',
    title='Loan Count by Poverty (MPI) Category'
)
fig.update_layout(height=400, showlegend=False)
fig.show()
```

The prior output highlights which MPI categories receive more loan activity by displaying a bar chart of loans broken down by poverty level.

5 Conclusion

5.0.1 Achievements

- Successfully analyzed over 586,000 Kiva loans, revealing key sectoral and geographic lending patterns.
- Identified agriculture as the largest sector with 166,832 loans totaling 119.7 million in loan amounts and 112 million funded (In Dollars).
- Highlighted the Philippines as the top country by loan volume (160,441 loans) with efficient funding durations averaging 9.3 days.
- Developed accurate predictive models achieving a Random Forest ROC AUC of 0.89 and funding duration RMSE of 12 days.
- Created an interactive dashboard summarizing median loan amounts (425 dollars), total funding (378 million), and enabling dynamic exploration of lending trends.

LO2 Achieved: Applied data science within the financial micro-lending domain through large-scale loan analysis, poverty-linked socioeconomic integration, predictive modeling, and sectoral/geographic financial insights. All analyses, models, and visualizations were designed to solve real challenges in global microfinance.

LO3 Achieved: Built a multi-dataset integrated analytical system that emulated internet-connected platforms by merging external Kiva databases, implementing automated pipelines, and developing an

interactive dashboard with real-time model predictions, reflecting the structure of modern database-driven financial systems.

5.0.2 Limitations

- Inability to fully integrate Multidimensional Poverty Index (MPI) data limited the depth of socioeconomic impact analysis.
- Some model features, like partner ID, may require further validation to ensure generalizability.
- Temporal aspects of loan dynamics were underexplored; funding duration averages may mask regional or temporal variability.
- Dependence on cleaned dataset integrity: missing or inconsistent data entries may bias some conclusions.

5.0.3 Future Work

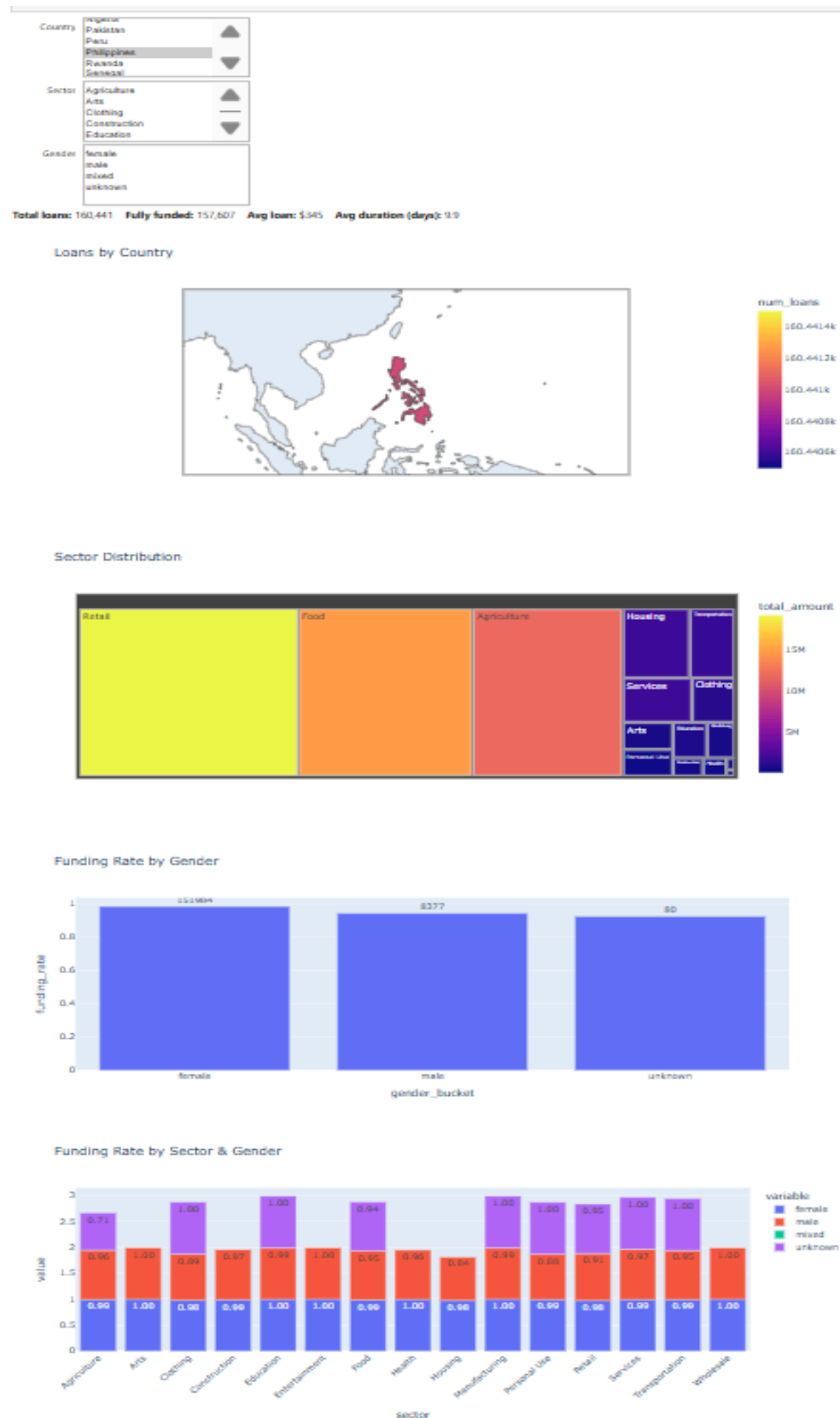
- Integrate comprehensive MPI data to enhance poverty targeting and impact measurement.
- Expand predictive modeling with temporal trend analysis and explainable AI to improve interpretability and robustness.
- Refine the dashboard with real-time updating and customized scenario analyses for stakeholders.
- Conduct longitudinal studies to better capture evolving lending patterns and socioeconomic effects.

[]:

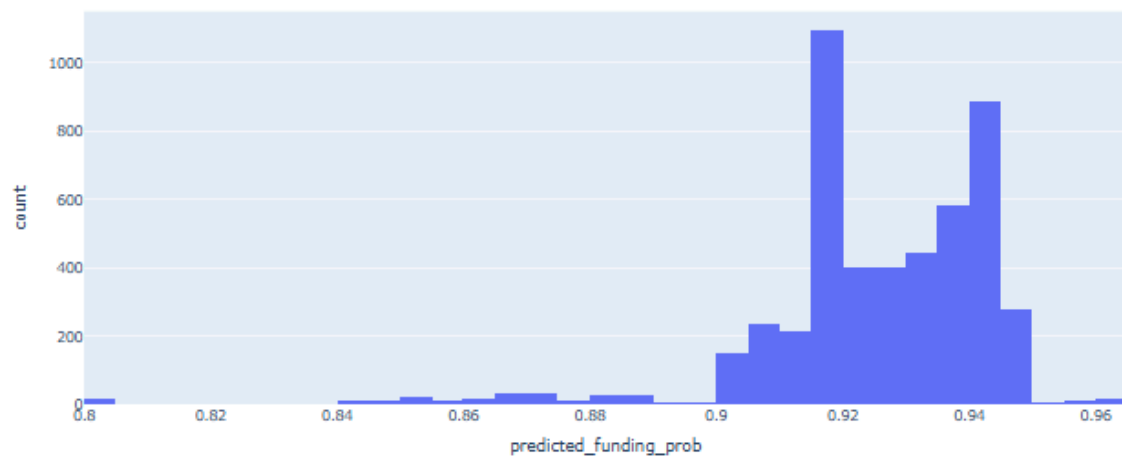
Objective 4

For showcase purposes, we have included a snippet of our interactive dashboard in the PDF. In this example, the country filter has been set to *Philippines and Kenya* to illustrate how the dashboard updates based on user selections.

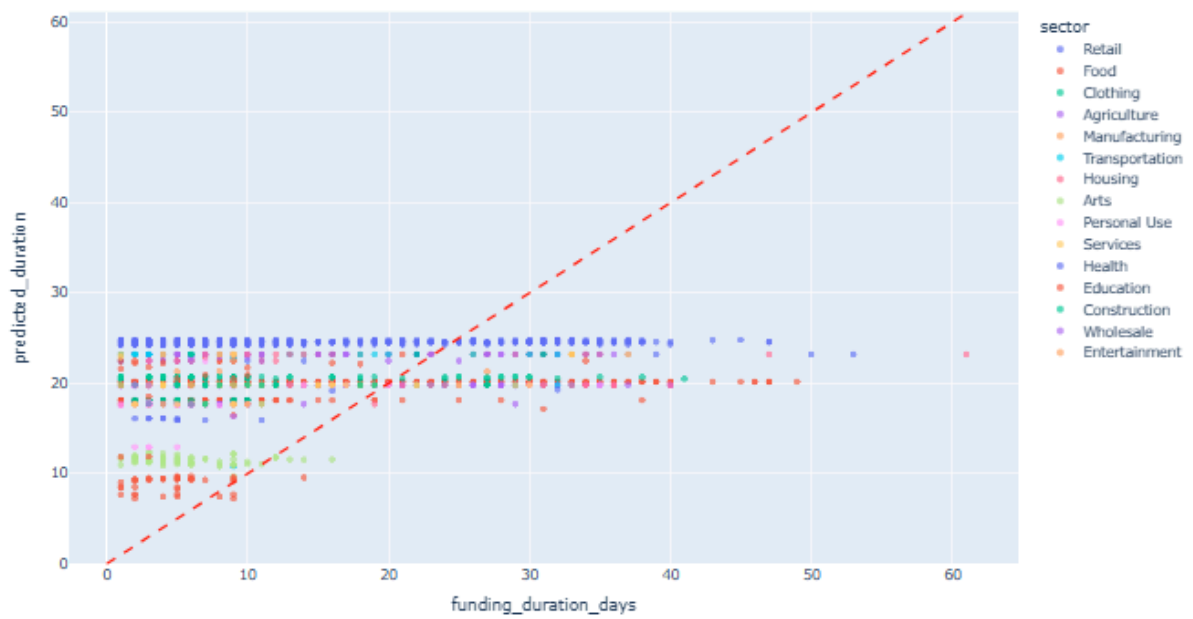
Dashboard visualisation for the Philippines



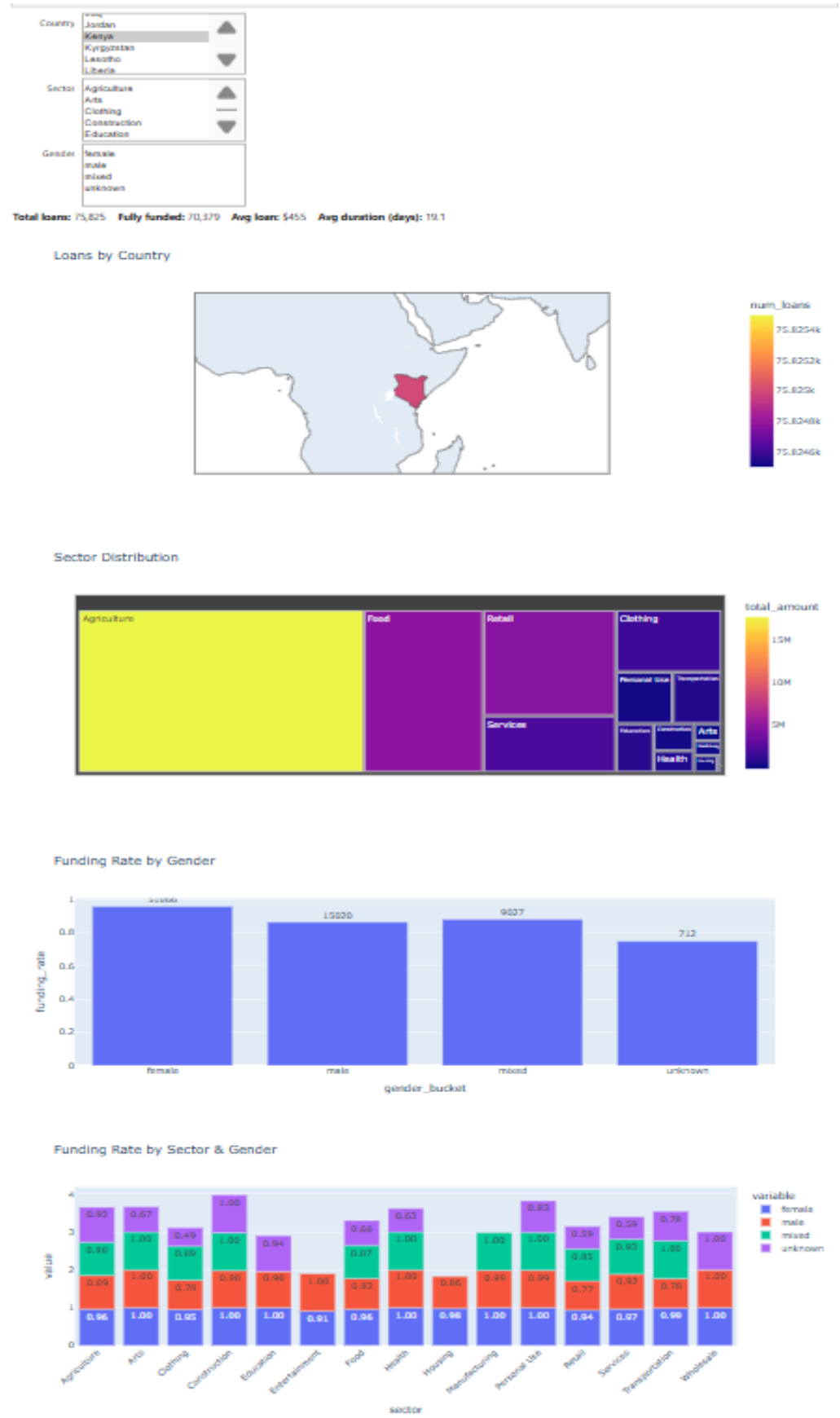
Predicted Funding Probability



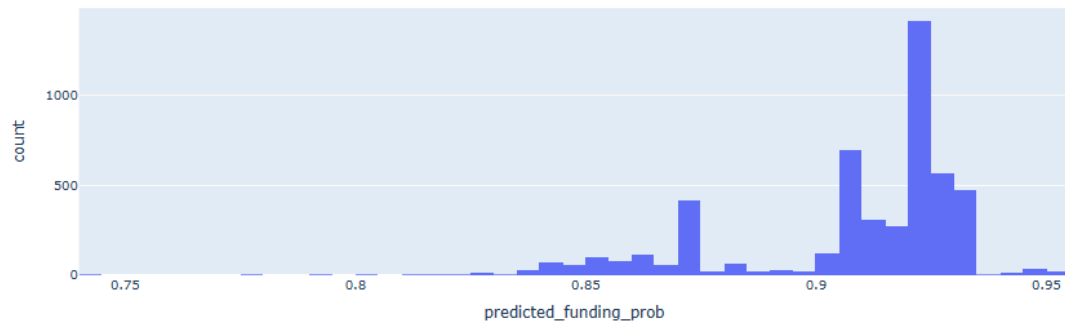
Actual vs Predicted Funding Duration



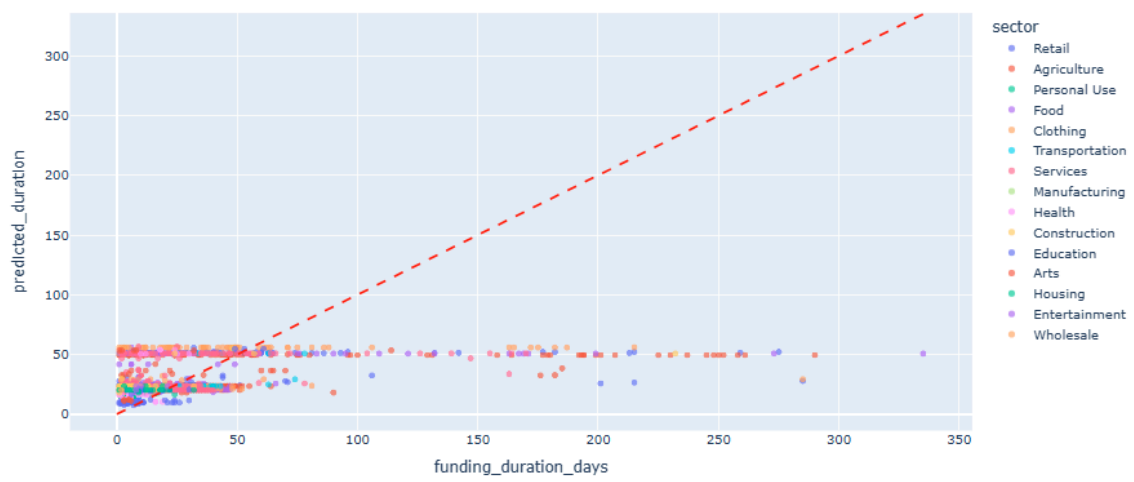
Dashboard visualisation for Kenya



Predicted Funding Probability

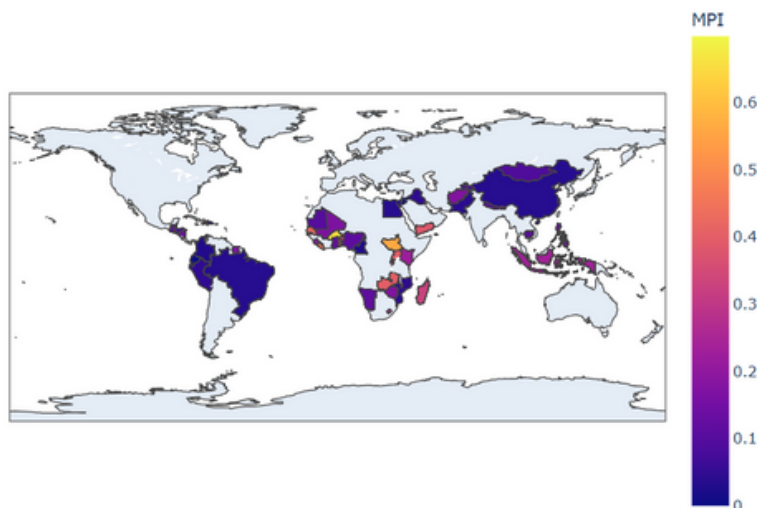


Actual vs Predicted Funding Duration

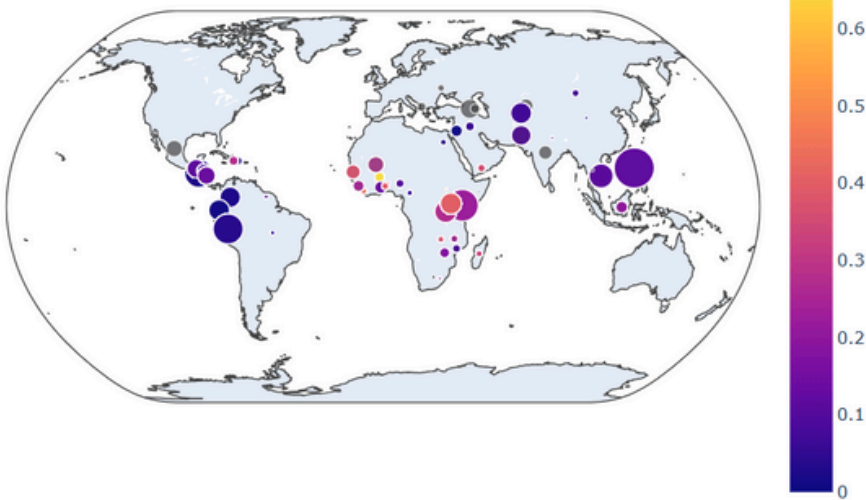


ADDITIONAL VISUALIZATION (due to error in export)

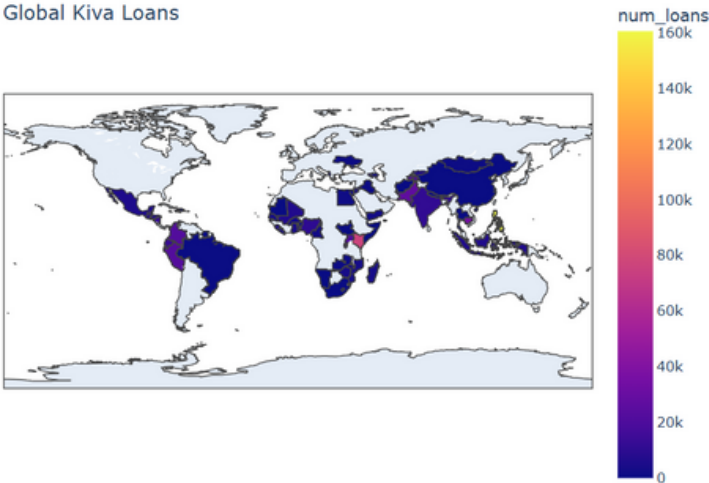
Global Poverty (MPI)



Loan Volume vs Poverty



Global Kiva Loans



Loans by Country and Sector

